Advanced QML:
Gradients, Cost, and Barren Plateaus

Lecture 7 — UZH Quantum Machine Learning Course

Dr. Pavel Sulimov
ZHAW School of Engineering

2026

Road map for today

From Classical to Quantum Gradients

The Parameter-Shift Rule

Barren Plateaus

Non-Gradient Methods

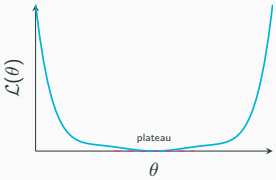
Quantum Advantage: When and How?

Putting It All Together

From Classical to Quantum Gradients

Why gradient-based optimisation rules deep learning

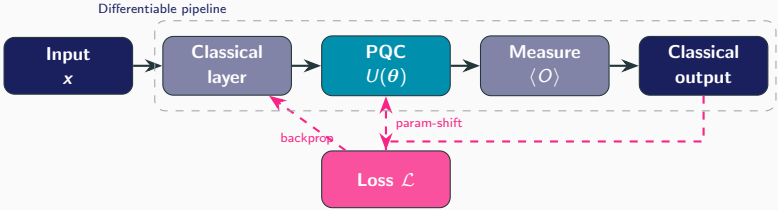
- Classical NN: one backward pass computes *all* gradients
  $\Rightarrow$ cost $\mathcal{O}(p)$ for p parameters
- Automatic differentiation (AD) chains the computation graph
- Enabled billion-parameter models



Key question for today

Can we do the same for **parametrised quantum circuits (PQC)**?

Hybrid quantum-classical architecture



Chain rule connects both worlds

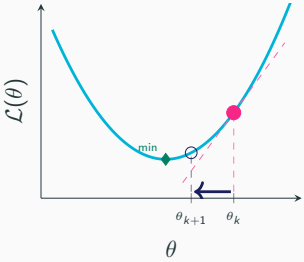
$$\frac{\partial \mathcal{L}}{\partial w} = \underbrace{\frac{\partial \mathcal{L}}{\partial \langle O \rangle}}_{\text{classical AD}} \cdot \underbrace{\frac{\partial \langle O \rangle}{\partial \theta}}_{\text{param-shift}} \cdot \underbrace{\frac{\partial \theta}{\partial w}}_{\text{classical AD}}$$

The Parameter-Shift Rule

Gradients in 60 seconds — why we need them

Intuition: a gradient tells you *which direction is downhill*.

$$\theta_{k+1} \leftarrow \theta_k - \eta \underbrace{\frac{\partial \mathcal{L}}{\partial \theta_k}}_{\text{“slope” at } \theta_k}$$



- Positive slope $\Rightarrow$ decrease θ_k
- Negative slope $\Rightarrow$ increase θ_k
- Zero slope $\Rightarrow$ stuck (local min or plateau)

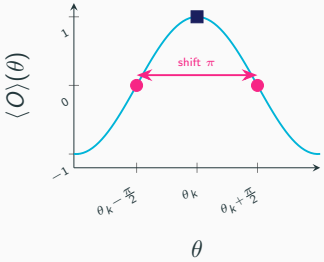
The catch for quantum circuits

You cannot read quantum states directly. **How do you even compute a slope?**
$\Rightarrow$ The parameter-shift rule.

The parameter-shift rule

For a gate $G(\theta) = e^{-i\frac{\theta}{2}P}$ where $P^2 = \mathbb{I}$:

$$\frac{\partial\langle O\rangle}{\partial\theta_k} = \frac{1}{2}\left[\langle O\rangle\left(\theta_k + \tfrac{\pi}{2}\right) - \langle O\rangle\left(\theta_k - \tfrac{\pi}{2}\right)\right]$$



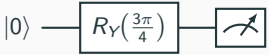
- **Exact** — no finite-difference approximation error
- Requires 2 circuit evaluations per parameter
- Cost for p parameters: **2p** evaluations

The parameter-shift rule — worked example

Setup: 1 qubit, $R_Y(\theta)\,|0\rangle$, observable $\hat{Z}$. Note: $\langle Z\rangle(\theta) = \cos\theta$ (exact identity).

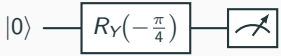
At $\theta_k = \frac{\pi}{4}$, run two shifted circuits:

Forward shift: $\theta_k + \frac{\pi}{2} = \frac{3\pi}{4}$



$$\langle Z\rangle_+ = \cos\tfrac{3\pi}{4} = -\tfrac{\sqrt{2}}{2} \approx -0.707$$

Backward shift: $\theta_k - \frac{\pi}{2} = -\frac{\pi}{4}$



$$\langle Z\rangle_- = \cos\left(-\tfrac{\pi}{4}\right) = +\tfrac{\sqrt{2}}{2} \approx +0.707$$

$$\left.\frac{\partial\langle Z\rangle}{\partial\theta}\right|_{\pi/4} = \tfrac{1}{2}\left(\langle Z\rangle_+ - \langle Z\rangle_-\right) = \tfrac{1}{2}\left(-0.707 - 0.707\right) = \mathbf{-0.707}$$

Cross-check: $\left.\frac{d}{d\theta}\cos\theta\right|_{\pi/4} = -\sin\frac{\pi}{4} = -\frac{\sqrt{2}}{2} \approx -0.707$ ✓

Gradient step: $\theta_{k+1} = \frac{\pi}{4} - \eta\cdot(-0.707)$ — positive correction, moves toward the minimum at $\theta^* = 0$.

"Can't we just backprop through the matrix multiplication?"

The intuition behind the question:

"$U(\boldsymbol{\theta})\,|\psi\rangle$ is just a matrix–vector product — differentiable like any PyTorch layer. Why not call `loss.backward()` directly?"

1. No-cloning theorem.
Backprop needs to save intermediate activations for the backward pass. Quantum states cannot be copied, so nothing to save.

2. Measurement collapses the state.
Reading the circuit output destroys it, so you cannot "replay" the forward pass in reverse.

3. Exponential classical cost.
Simulating $U(\boldsymbol{\theta})\,|\psi\rangle$ classically requires a 2^n-entry state vector. Backpropagating through it costs $\mathcal{O}(2^n)$ memory.

What parameter-shift gives instead

Only *expectation values* are needed — no state access, no exponential memory. Each term is a plain hardware measurement — no internal state needs to be stored or read.

Classical simulation: the special case

On a **simulator** (e.g. `default.qubit`), classical backprop *is* available: `diff_method="backprop"`. It is fast and convenient, but costs $\mathcal{O}(2^n)$ memory.

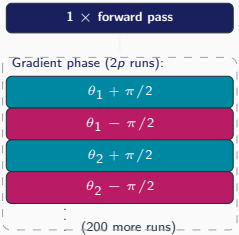
Why quantum gradient computation is expensive

Quantum states cannot be read analytically—we sample:

$$\langle O \rangle \approx \frac{1}{S} \sum_{s=1}^{S} m_s, \quad \text{std. error} \sim \frac{1}{\sqrt{S}}$$

Concrete example: 100-parameter PQC

Stage	Circuit runs
Forward pass	1
Gradient (param-shift)	$2 \times 100 = 200$
Total	**201**
$\times$ 1000 shots/run	**201 000 measurements**



Beyond parameter-shift: smarter gradient strategies

SPSA (Simultaneous Perturbation Stochastic Approximation)

$$\hat{g}_k = \frac{\mathcal{L}(\boldsymbol{\theta} + c\boldsymbol{\delta}) - \mathcal{L}(\boldsymbol{\theta} - c\boldsymbol{\delta})}{2c\boldsymbol{\delta}}$$

$\boldsymbol{\delta} \sim$ Rademacher $\{\pm 1\}^p$

Only 2 runs regardless of p; trade-off: noisier gradients.

Quantum Natural Gradient (QNG)

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta\, \mathcal{F}^{-1}(\boldsymbol{\theta})\, \nabla \mathcal{L}$$

$\mathcal{F} =$ Quantum Fisher Information Matrix

Accounts for the *geometry* of quantum state space; fewer optimisation steps needed.

Method	Runs	Exact?	Noisy?	Cost
Param-shift	$2p+1$	**Yes**	OK	$\mathcal{O}(p)$
Finite diff.	$p+1$	Approx.	Sensitive	$\mathcal{O}(p)$
SPSA	2	Approx.	**Good**	$\mathcal{O}(1)$
QNG	$2p^2$	**Yes**	Sensitive	$\mathcal{O}(p^2)$
Adjoint	1	**Yes**	Sim. only	$\mathcal{O}(1)$

PennyLane primer — for students who know Qiskit

Qiskit → PennyLane concept map:

Qiskit	PennyLane
`QuantumCircuit`	`@qml.qnode`
`execute(circ, backend)`	call like Python function
`ParameterVector`	plain `np.array`
Manual gradient est.	`qml.grad(f)(params)`
`AerSimulator`	`qml.device(...)`

Three objects you need

1. `qml.device(...)`
   backend: simulator or hardware
2. `@qml.qnode(dev, diff_method=...)`
   wraps circuit into differentiable fn
3. `qml.grad(f)`
   returns $\nabla_\theta f(\theta)$ via param-shift

Mental shift

Qiskit: build ⇒ transpile ⇒ run
PennyLane: define ⇒ differentiate ⇒ optimise
`@qml.qnode` ≈ PyTorch `nn.Module` for quantum.

Why not just Qiskit?

No built-in param-shift autodiff. PennyLane makes gradient-based QML the *default*.

Hands-on: gradients in PennyLane

```python
import pennylane as qml
import pennylane.numpy as np  # PennyLane's numpy wrapper

dev = qml.device("default.qubit", wires=2)

@qml.qnode(dev, diff_method="parameter-shift")
def circuit(params):
    qml.RY(params[0], wires=0)
    qml.CNOT(wires=[0, 1])
    qml.RY(params[1], wires=1)
    return qml.expval(qml.PauliZ(0))

# requires_grad=True tells PennyLane which params to differentiate
params = np.array([0.5, 0.3], requires_grad=True)
grad_fn = qml.grad(circuit)
print(grad_fn(params))        # [L/, L/]

# SPSA | only 2 circuit runs regardless of parameter count!
@qml.qnode(dev, diff_method="spsa", num_directions=1)
def circuit_spsa(params): ...
```

pennylane.ai/qml/glossary/parameter_shift

Accelerating gradients with PennyLane Catalyst

Without Catalyst — Python overhead on every call

```python
# Standard: Python re-traces every call
import pennylane.numpy as np
dev = qml.device("default.qubit", wires=n)

@qml.qnode(dev, diff_method="parameter-shift")
def circuit(params): ...

grad_fn = qml.grad(circuit)
for step in range(1000):
    g = grad_fn(params)  # slow: Python loop
```

With Catalyst — compile once, run fast

```python
# pip install pennylane-catalyst
import catalyst
import numpy as np  # plain numpy with Catalyst
dev = qml.device("lightning.qubit", wires=n)

@qml.qjit                # JIT-compile everything
@qml.qnode(dev)
def circuit(params): ...

grad_fn = catalyst.grad(circuit)
for step in range(1000):
    g = grad_fn(params)  # fast: native code
```

What `@qml.qjit` compiles

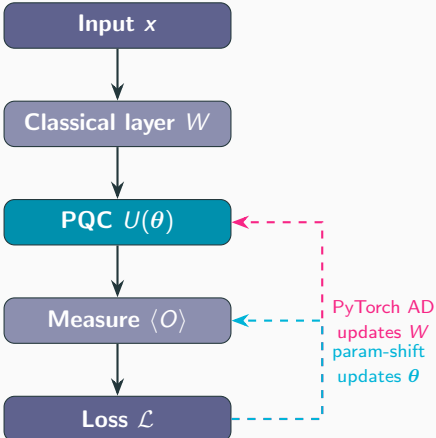
- **Forward pass** — circuit + measurement → MLIR/LLVM native
- **Backward pass** — `catalyst.grad` differentiates through the compiled pipeline (no Python re-entry)
- **Full loop** — wrap optimizer step inside `@qml.qjit`

Key differences from standard PL

- Device: `lightning.qubit` (not `default.qubit`)
- Use plain `numpy`, not `pennylane.numpy`
- `catalyst.grad` instead of `qml.grad`
- Typical speedup: 5–50× on CPU

Hybrid training: quantum and classical in one pass

The picture: a classical layer feeds into a PQC, feeds into a loss. We want to update *both* sets of weights in one backward pass.



How it works mechanically:

Setting `interface="torch"` on a `@qml.qnode` tells PennyLane to register the parameter-shift gradient as a `torch.autograd.Function`.

From PyTorch's perspective the QNode is just another differentiable layer. Calling `loss.backward()` automatically chains through it:

- Classical parameters W — updated via standard PyTorch reverse-mode AD
- Quantum parameters $\boldsymbol{\theta}$ — updated via parameter-shift, transparently called inside the autograd graph

Hybrid training: PennyLane + PyTorch in code

```python
import torch
import pennylane as qml
dev = qml.device("default.qubit", wires=2)
# interface="torch" registers param-shift as a PyTorch autograd.Function
@qml.qnode(dev, interface="torch", diff_method="parameter-shift")
def circuit(params):
    qml.RY(params[0], wires=0)
    qml.CNOT(wires=[0, 1])
    return qml.expval(qml.PauliZ(0))
classical = torch.nn.Linear(4, 2)
q_params  = torch.tensor([0.5, 0.3], requires_grad=True)
# Classical AND quantum parameters in a single optimizer
opt = torch.optim.Adam(list(classical.parameters()) + [q_params])
x      = torch.randn(4)
out_cl = classical(x)           # classical forward pass
out_q  = circuit(out_cl)        # quantum forward pass
loss   = (out_q - torch.tensor(1.0))**2
loss.backward()   # one call: PyTorch AD flows into param-shift at the QNode
opt.step()        # updates both W and theta simultaneously
```

Barren Plateaus

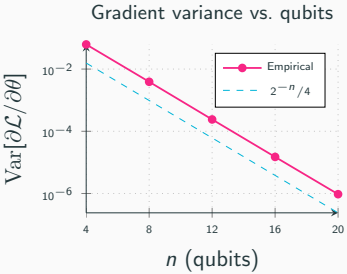
What are barren plateaus?

Definition (McClean et al. 2018)

Gradients of a randomly-initialised PQC vanish *exponentially* in the number of qubits n:

$$\mathbb{E}\left[\frac{\partial \mathcal{L}}{\partial \theta_k}\right] = 0, \quad \mathrm{Var}\left[\frac{\partial \mathcal{L}}{\partial \theta_k}\right] \in \mathcal{O}\left(\frac{1}{2^n}\right)$$

- Not a local minimum — the landscape is globally flat
- Affects *most* random starting points
- The gradient is so small that any gradient step is essentially *random noise* — you can't tell which direction is downhill



Reading the barren plateau definition

Two separate statements:

1. $\mathbb{E}\left[\partial\mathcal{L}/\partial\theta_k\right] = 0$

   Average over random inits is zero — no directional signal for the optimiser.

2. $\mathrm{Var}\left[\partial\mathcal{L}/\partial\theta_k\right] \in \mathcal{O}(2^{-n})$

   Spread collapses to $\sigma \sim 2^{-n/2}$ — indistinguishable from shot noise.

Architecture vs. data?

Driven by *circuit expressibility* + *observable scope* (global vs. local — details next), **not** the dataset. Diagnose via gradient variance vs. n; no data needed.

Classical vs. quantum fix

Classical: swap activation (ReLU). Quantum: gates are unitary — the lever is the *observable* (local cost function).

Classical parallel: vanishing gradients

Quantum (BP)	Classical (deep NN)
why it happens:	
More qubits n	More layers L
High expressibility	Sigmoid activation
$\sigma \sim 2^{-n/2}$	$\sigma \sim \lambda^{-L}$, $\lambda < 1$
Random init	Random init
how to fix (next slides):	
Local cost fn.	ReLU / batch norm
Identity-block init	Xavier / He init
Layer-wise training	Residual connections

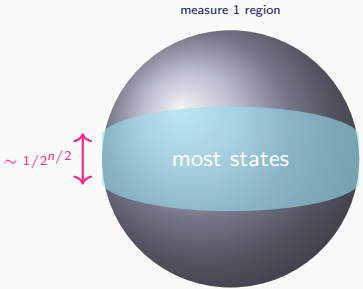
Intuition: concentration of measure

High-dimensional geometry

Random unitaries on n qubits explore a Hilbert space of dimension 2^n. A random state concentrates near the "equator" — the output distribution becomes nearly uniform, and almost every observable has the *same* expectation.

Intuitive analogy

Imagine picking a random point on a very high-dimensional sphere. Almost surely you land near the "equatorial belt" — any function that depends on which hemisphere you're on varies almost not at all across typical points.



Diagnosing barren plateaus

Method 1: gradient variance scaling

Train identical architectures with $n = 4, 6, 8, \ldots$ qubits; track

$$\hat{\sigma}_k^2 = \frac{1}{M} \sum_{m=1}^{M} \left(\frac{\partial \mathcal{L}}{\partial \theta_k^{(m)}} \right)^2$$

If $\hat{\sigma}_k^2 \propto 2^{-n} \Rightarrow$ barren plateau.

Method 2: loss plateau detection

Monitor the *effective gradient norm* during training:

$$\|\nabla \mathcal{L}\|_\infty < \varepsilon_{\text{plateau}}$$

and check whether loss $\mathcal{L}$ stagnates *before* reaching a minimum.

Method 3: expressibility diagnostic

High circuit expressibility $\Leftrightarrow$ high barren-plateau risk (Holmes et al., 2022).

"How much" are we on a plateau?

Define a **plateau severity index**: $\mathcal{P} = 1 - \dfrac{\hat{\sigma}_k^2(n)}{\hat{\sigma}_k^2(n_0)}$ normalised to the reference scale n_0.

$\mathcal{P} \to 1$ signals full exponential suppression.

Escaping barren plateaus: a toolkit

Initialisation strategies

- **Identity block init** — start near identity, so gradients are large
- **Problem-inspired init** — warm-start with a classical solver or QAOA
- **Layer-by-layer training** — fix earlier layers, train new ones incrementally

Architecture constraints

- **Local cost functions** — measure k-qubit subsystems: $\mathcal{L}_{\text{local}} = \sum_i \langle Z_i \rangle$
- **Shallow circuits** — restrict depth $\leq \mathcal{O}(n)$

Optimiser-level tricks

- **Correlated parameter perturbations** (SPSA, CMA-ES)
- **Quantum natural gradient** — follows curved geometry of state space
- **Rotosolve / Rotoselect** — analytic optimisation per parameter in trigonometric landscapes

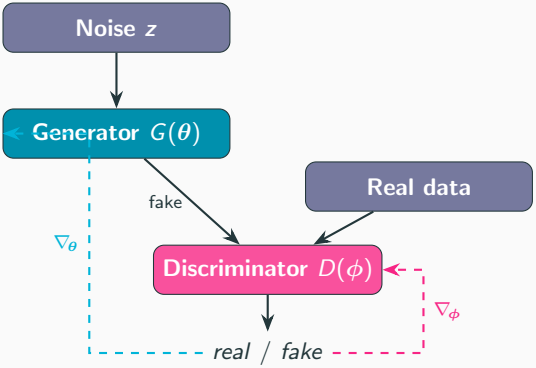
Key insight

Local cost functions reduce plateau severity from $\mathcal{O}(2^{-n})$ to $\mathcal{O}(\mathrm{poly}(n)^{-1})$ (Cerezo et al., 2021).

Quantum Generative Adversarial Networks (qGANs)

Classical GAN idea, lifted to quantum circuits:

- **Generator** $G(\boldsymbol{\theta})$: a PQC that maps random noise $\boldsymbol{z}$ to a quantum state intended to mimic real data.
- **Discriminator** $D(\boldsymbol{\phi})$: another PQC (or classical net) that tries to tell real samples from generated ones.
- They play a **minimax game**: G wants to fool D; D wants to catch G. Both are trained simultaneously with gradients.



qGANs: why adversarial training mitigates barren plateaus

Three structural reasons the adversarial setup helps:

1. **Discriminator output is a local cost.**
   D outputs a scalar (real/fake probability), not a global Hamiltonian expectation — functionally equivalent to a local observable, so gradient decay is polynomial.

2. **Adaptive training signal.**
   As G improves, D must work harder to discriminate; the loss landscape keeps shifting, preventing any permanent gradient freeze.

3. **Non-zero gradients by construction.**
   While G and D are in disequilibrium — most of training — both have non-trivial gradient signal coming from the other network.

Not fully immune

If D overtakes G too quickly, the generator loss saturates and gradients do vanish. Standard mitigation (local cost, shallow circuits, identity-block init) still applies.

References

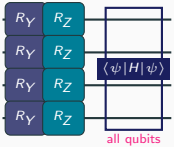
Lloyd & Weedbrook,
PRL 121, 040502 (2018)

Dallaire-Demers & Killoran,
PRA 98, 012324 (2018)

Braccia et al.,
Quantum 5, 602 (2022)

Local vs. global cost functions

Global cost

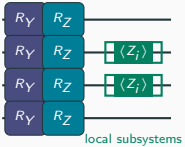


Gradient $\sim 2^{-n}$

exponential suppression

$\Rightarrow$

Local cost (preferred)



Gradient $\sim n^{-c}$

polynomial suppression only

Why does this work?

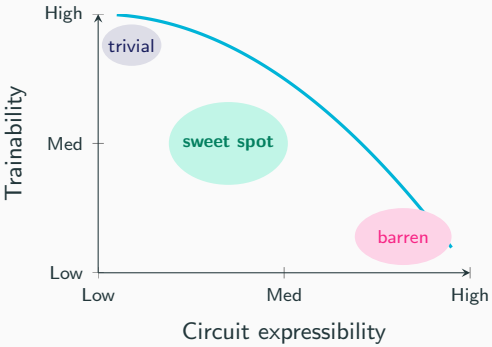
A local observable on k qubits only "sees" a 2^k-dimensional subspace, avoiding the exponential concentration of the full 2^n-dimensional state space. Fewer dimensions $\Rightarrow$ gradients don't wash out.

Classical analogy?

Not regularisation (L2/dropout) — that fights *overfitting*; this fights *gradient vanishing* (different axes: optimisation vs. generalisation).

Closest parallel: **auxiliary losses** (e.g. GoogLeNet 2014) — local supervision per layer to keep gradients alive through depth.

The expressibility–trainability trade-off



- **High expressibility**
  ⇒ better function approximation
  ⇒ exponentially harder to train
- **Low expressibility**
  ⇒ easy to train
  ⇒ limited problem-solving power
- **Target the sweet spot**:
  moderate entanglement,
  problem-informed architecture

Connection to universality: the third rotation axis

A key result from single-qubit universality:

Any single-qubit gate decomposes as

$$U = e^{i\alpha} R_x(\beta)\, R_y(\gamma)\, R_z(\delta)$$

- **State prep** from $|0\rangle$: only $R_x R_y$ needed — two axes already cover the full Bloch sphere.
- **General gate** on any input: the last $R_z(\delta)$ sets a *relative phase* between $|0\rangle$ and $|1\rangle$ columns — it matters for superposition inputs.

Why this matters for barren plateaus

Using $R_x R_y R_z$ per qubit in your ansatz $\Rightarrow$ **maximum expressibility** $\Rightarrow$ circuit looks like a Haar-random unitary $\Rightarrow$ exponential barren plateau.

Practical design rule

Hardware-efficient ansätze use only R_y (or $R_x + R_y$) per qubit, intentionally *sacrificing* the third axis.

- Less expressive per layer
- **Trainable gradients**
- Compensate with entanglement and more layers

Non-Gradient Methods

When gradients fail: non-gradient optimisation

Derivative-free methods

- **COBYLA** — VQE workhorse; few evaluations; noise-tolerant
- **CMA-ES** — adapts search covariance; effective for $p \lesssim 200$
- **Nelder-Mead / Powell** — simplex/direction-set; early NISQ
- **Bayesian optimisation** — surrogate model; expensive landscapes

Rotosolve: analytic optimisation per parameter

Loss for a 1-parameter rotation gate is *exactly*:

$$\mathcal{L}(\theta_k) = A \sin(\theta_k + \phi) + C$$

Three evaluations $\Rightarrow$ exact minimum. Cycle over all θ_k: **Rotosolve** (Ostaszewski 2021).

When to prefer non-gradient methods

- Noisy hardware (gradient estimation unreliable)
- Small circuits ($p < 50$)
- Barren plateau detected

Quantum Advantage: When and How?

Quantum advantage in QML — the honest picture

The hard truth

Classical NNs often match NISQ models on the same task. **Quantum advantage is not automatic.**

Where theoretical advantage exists:

- **Quantum chemistry / simulation** — native Hamiltonian encoding
- **Quantum kernel methods** — classically-hard feature maps
- **Specific learning tasks** — quantum phase recognition (Liu 2021)

How proper training helps

1. **Avoid barren plateaus** — ensure genuine convergence
2. **Problem-informed ansätze** — respect target symmetries
3. **Honest benchmarking** — compare at equal parameter count

Practical guidance

Advantage is most credible when: (a) training loss is genuinely low, (b) gradient-informed training was used, (c) problem has known quantum structure.

The parameter-count fallacy

⚠ A common but flawed claim

"Our QM model has 24 parameters vs. 10 000 classical — so it generalises better and overfits less."

Why this comparison is misleading:

- One gate $R_Y(\theta)$ acts on a 2^n-dimensional Hilbert space — effective expressivity $\gg$ one scalar d.o.f.
- Classical and quantum parameters are incommensurable — comparing them is like comparing apples to qubits
- Training dynamics differ: barren plateaus, shot noise, hardware error — none scale with parameter count

What actually matters

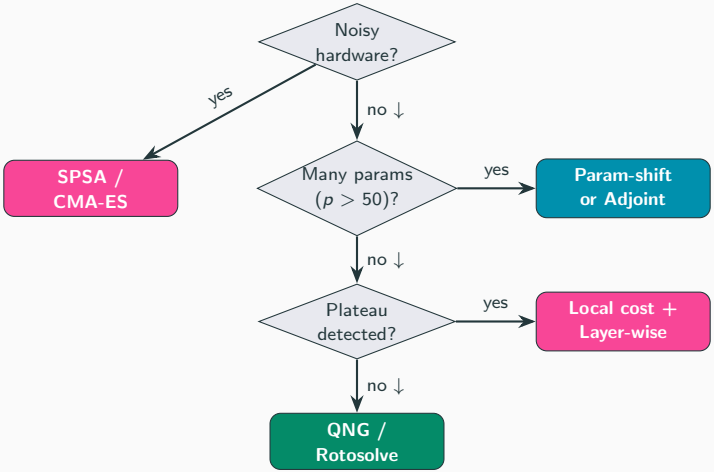
1. **Validation loss curve** — not training loss, not parameter count: does the model generalise to unseen data?
2. **Convergence cost** — gradient steps $\times$ circuit runs $\times$ shots; quantum is often orders of magnitude pricier
3. **Sample complexity** — how many data points until validation loss plateaus?

Red flags in quantum ML papers

- No validation curve shown
- Classical baseline has $10\times$ more parameters *by design*
- "Fewer parameters" cited as advantage without convergence data

Putting It All Together

Decision tree: choosing your optimisation strategy



Take-aways

1. **Parameter-shift rule** gives *exact* quantum gradients at cost $\mathcal{O}(2p)$. **SPSA** cuts this to $\mathcal{O}(1)$ with some noise.
2. **Barren plateaus**: gradients scale as 2^{-n} with random init + high expressibility — deep PQCs are untrainable by default.
3. **Diagnose** via gradient-variance scaling; **quantify** with severity index $\mathcal{P} = 1 - \hat{\sigma}^2(n)/\hat{\sigma}^2(n_0)$.
4. **Escape**: local cost functions, identity-block init, layer-wise training, QNG, Rotosolve.
5. The **expressibility–trainability trade-off** is fundamental — $R_z R_y R_z$ per qubit maximises expressibility but invites barren plateaus; R_y-only often hits the sweet spot.

`pennylane` $\geq$ `0.44` — McClean (2018) · Cerezo (2021) · Ostaszewski (2021)

Appendix

Appendix: deriving the parameter-shift rule

Gate family: $G(\theta) = e^{-i\frac{\theta}{2}P}$, $P^2 = \mathbb{I}$.
Expectation: $f(\theta) = \langle\psi|\, G^\dagger(\theta)\, O\, G(\theta)\, |\psi\rangle$.

$$f(\theta) = \langle\psi|\left(\cos\tfrac{\theta}{2}\,\mathbb{I} + i\sin\tfrac{\theta}{2}\,P\right) O \left(\cos\tfrac{\theta}{2}\,\mathbb{I} - i\sin\tfrac{\theta}{2}\,P\right)|\psi\rangle$$

$$\frac{df}{d\theta} = \tfrac{1}{2}\left[f\left(\theta + \tfrac{\pi}{2}\right) - f\left(\theta - \tfrac{\pi}{2}\right)\right]$$

The identity $\cos(x \pm \pi/4) = \mp\sin(x)$ turns the derivative into a **finite difference at shifts** $\pm\pi/2$ — no approximation needed.

Appendix: Quantum Fisher Information

$$\mathcal{F}_{ij}(\boldsymbol{\theta}) = 4\,\mathrm{Re}[\langle \partial_i \psi | \, \partial_j \psi \rangle - \langle \partial_i \psi | \, \psi \rangle \, \langle \psi | \, \partial_j \psi \rangle]$$

- Measures how much the quantum state $|\psi(\boldsymbol{\theta})\rangle$ changes with parameters θ_i, θ_j.
- QNG update is the steepest-descent step in the *Bures metric* on state space, not Euclidean parameter space.
- Practically: approximate $\mathcal{F}$ with the **block-diagonal** approximation to reduce the $\mathcal{O}(p^2)$ cost.